Small Updates Break First: A Pre-Registered Study of Fine-Tuning Gains Under MLX and GGUF Quantization

Jayesh Suryavanshi *jayeshksuryavanshi@gmail.com*

Abstract

Small language models are often fine-tuned and then shipped at 3 to 6 bits per weight, as MLX affine or GGUF k-quant files, on the assumption that the fine-tuning gain survives. We tested that assumption in six pre-registered rounds on one laptop. We fine-tuned Qwen3 at 0.6B, 1.7B and 4B parameters and OLMo-2 1B with LoRA (and, on Qwen3-0.6B, full fine-tuning) for 500 steps on banking77 intent classification and MASSIVE slot annotation, evaluated the models on their full test sets at bf16 and up to ten quantized formats, and measured gain retention R, the share of a fine-tune's bf16 advantage over its base model that survives when both are quantized. In all 10 pre-registered tests (five fine-tune pairs, each at MLX 3-bit and GGUF Q3_K_M), a LoRA trained at learning rate 10^{-4} retained more of its gain than the same LoRA at 10^{-5}, mlx-lm's default learning rate, by +0.06 to +0.63 in R; with mlx-lm's other defaults, the gap at Q3_K_M was +0.48, but at MLX 3-bit both LoRAs lost almost all of their gain. In exploratory comparisons over eight settings and five formats, the 10^{-5} LoRA kept a smaller share of its own accuracy in 40 of 40 cases, yet remained the more accurate model at Q4_K_M in all eight settings, while the 10^{-4} LoRA was the more accurate one at MLX 3-bit and Q2_K in all eight. The loss does not come from rounding the update. Keeping the 10^{-5} update exact on the quantized base recovered 5% and 12% of the lost gain in MLX (OLMo-2 1B, 3-bit, two seeds; upper 95% bounds 16% and 22%) and, post hoc, did worse than fusing in GGUF (Qwen3-0.6B, Q3_K_M), while training a 10^{-5} Qwen3-0.6B LoRA against the weights that ship delivered 96% of the bf16 gain at MLX 4-bit and 99% at Q3_K_M, against 84% and 79% when it was trained in bf16, fused and quantized. At 10^{-4} the same step cost 0.158 and 0.115 of R (exploratory). An importance matrix narrowed the gap between the two learning rates without closing it (pre-registered test inconclusive). A three-parameter predictor built from the weights and a one-time measurement of the base model met its pre-registered error bar of 0.10 on held-out fine-tunes (mean absolute error 0.059 and 0.084), but a post hoc model of update size and base damage reached 0.049 in both rounds.

1 Introduction

Fine-tuning a small open model and quantizing it for local inference is a common way to ship a task-specific language model. The artifact is usually an MLX model for Apple silicon (Hannun et al., 2023) or a GGUF file for llama.cpp (Gerganov & contributors, 2023) and the tools built on it, at 3 to 6 bits per weight. Evaluations of quantized models, including GGUF k-quants (Kurt, 2026), mostly measure released base or instruction-tuned models on broad benchmarks (Kurtic et al., 2025; Kurt, 2026), and rarely hold a task fine-tune fixed while varying the format. There are signs that a fine-tune does not always keep what it learned. Quantizing after parameter-efficient fine-tuning loses accuracy at low bit widths (Xu et al., 2023), quantizing an unlearned model restores knowledge that small-learning-rate unlearning had removed (Zhang et al., 2025), and merging a LoRA adapter into a native 4-bit checkpoint and writing it back through the quantizer can delete the adaptation (Li et al., 2026).

This paper asks a narrower question with controlled evidence. When a fine-tuned model is quantized with calibration-free MLX and GGUF formats, how much of the fine-tuning gain survives, which fine-tunes lose it, why, and can the loss be predicted from the weights before quantizing? We answered it in six rounds

of experiments, each pre-registered and hashed before its confirmatory runs, on one Apple M1 Pro laptop with 16 GB of memory. The study covers Qwen3 at 0.6B, 1.7B and 4B parameters (Yang et al., 2025) and OLMo-2 1B (Team OLMo, 2025); banking77 intent classification (Casanueva et al., 2020) and MASSIVE slot annotation (FitzGerald et al., 2023); LoRA (Hu et al., 2022) at learning rates from 3×10^{-6} to 3×10^{-4}, and full fine-tuning on Qwen3-0.6B; and MLX affine quantization from 8 to 2 bits and GGUF quantization from Q8_0 to Q2_K, with and without an importance matrix. Every fine-tune of rounds 1 to 4 was evaluated on its complete test set in both engines. We call a LoRA trained at learning rate 10^{-5} gentle and one trained at 10^{-4} strong; the gentle learning rate is mlx-lm's default, and in rounds 1 to 4 its update was 8.7 to 12.9 times smaller. Table 1 states each finding with the quantity measured, its estimate and the evidence behind it, and `ftquant check` computes the weight measurements for a user's own fine-tune with numpy on a CPU (Appendix E).

Every pre-registered result is reported, including the hypotheses that were not supported and the criterion the first predictor failed. Seven deviations from the pre-registered plans are listed in Appendix A.

2 Related work

Post-training quantization. LLM.int8() (Dettmers et al., 2022), GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) showed that large language models can be quantized to 8 and 4 bits with little loss on standard benchmarks. Local inference stacks mostly use simpler formats that need no calibration data: MLX's affine quantization, with a scale and a bias per group of weights (Hannun et al., 2023), and llama.cpp's GGUF block formats, including the mixed-precision k-quants (Gerganov & contributors, 2023). We study these files without calibration data, except in round 6's test of llama.cpp's importance matrix, which uses sample text to reduce quantization loss (Section 6.6).

Fine-tuning meets quantization. LoRA (Hu et al., 2022) trains a low-rank update that can be merged into the base weights. Because quantizing after fine-tuning loses accuracy at low bit widths, several methods fine-tune against the quantized model instead. QLoRA (Dettmers et al., 2023) trains the update on top of a 4-bit base, QA-LoRA (Xu et al., 2023) lets the merged model stay quantized, and LoftQ (Li et al., 2023) initializes the adapter to offset the gap between the quantized and full-precision base. BitDelta (Liu et al., 2024) instead stores a fine-tune as a separate 1-bit delta. The closest precedent for our results comes from unlearning. Zhang et al. (2025) found that quantizing a model after unlearning restores much of the forgotten knowledge: unlearning uses small learning rates, so quantization maps the original and the unlearned weights to the same values, and they recommend a larger learning rate. Attacks use the same arithmetic in the other direction, planting behavior that appears only after quantization (Egashira et al., 2024; 2025). Li et al. (2026) show that on native 4-bit microscaling checkpoints (NVFP4, MXFP4), naively merging and re-quantizing deletes the adaptation, by up to 39 points, because against an already on-grid base the reconstruction optimum is the base itself; they avoid this by adapting only the per-block scales. In mlx-lm, fusing an adapter into a pre-quantized gpt-oss-20b gave a model with 0% task success while the unfused adapter worked, although the reporter saw the same failure after fusing into a dequantized fp16 model (proflouishyman, 2026); a proposed fix, closed without merging, blamed LoRA deltas far below the 4-bit quantization step and advised training larger updates or shipping more bits (vaibzvb, 2026). We study the more common pipeline of fusing in bf16 and quantizing afterwards, for fine-tuning gains rather than unlearning: we test the larger-update advice with a controlled learning-rate sweep and pre-registered replications on the MLX and GGUF files people ship, and test whether rounding of the update is what causes the loss (Sections 6.5 and 6.6).

Predicting what quantization breaks. Accuracy can hide how much a compressed model changes: Dutta et al. (2024) show that answers flip in both directions while accuracy stays flat, and argue for distance measures such as KL divergence; our base-damage measure is one. In pretraining, Catalan-Tatjer et al. (2025) find that the learning rate and its schedule shape how well a model survives post-training quantization. Wu et al. (2026) predict flip rates from how decision margins on task inputs shrink under quantization, with a median error of 1.0 percentage point across 1,154 combinations of models and settings; ours uses only the weights and a one-time measurement of the base model, and a comparison with theirs remains open.

Table 1: Findings. R is gain retention and A accuracy retention (Section 3); gentle and strong are LoRAs at learning rates 10^{-5} and 10^{-4}; cells are setting and format pairs. Brackets are 95% paired bootstrap intervals over test items, except for MAE, whose intervals resample fine-tunes. Every value is recomputed from the per-item predictions (Appendix F).

	Finding	Measure and result	Evidence
F1	At MLX 3-bit and Q3_K_M, the gentle LoRA keeps less of its gain than the strong one	5 pairs on 3 base models and 2 tasks: $R_{\text{strong}} - R_{\text{gentle}}$ from +0.06 to +0.63, interval above 0 in 10 of 10 tests, also at the 99.5% level; with mlx-lm's other defaults +0.479 [+0.449, +0.509] at Q3_K_M, but at MLX 3-bit both lost almost all of their gain (exploratory)	Pre-registered, supported (W3-H1, W4-H1, W4-H2; with mlx-lm's defaults, W6-H3); §6.4, §6.6
F2	The gentle LoRA keeps a smaller share of its accuracy at every format compared	$A_{\text{strong}} - A_{\text{gentle}} > 0$ in 40 of 40 cells: at least 3.8 points at MLX 4-bit and 4.5 at MLX 3-bit, Q3_K_M and Q2_K; under 2 points in 6 of 8 at Q4_K_M	Exploratory; §6.4
F3	Which LoRA is more accurate after quantization depends on the format	Strong more accurate in 8 of 8 settings at MLX 3-bit, 8 at Q2_K, 7 at Q3_K_M, 5 at MLX 4-bit and 0 at Q4_K_M; its bf16 cost: 1.5 to 9.9 points	Exploratory; §6.4
F4	Keeping a gentle update exact does not restore its gain	MLX, OLMo-2 1B at 3-bit, 2 seeds: unfused minus fused R +0.008 [−0.010, +0.027] and +0.023 [+0.002, +0.042], upper bounds 16% and 22% of the lost gain; GGUF, Qwen3-0.6B at Q3_K_M: adapter at load time 0.733 against fused 0.795, 0.164 [0.146, 0.183] short of halfway to 1	Pre-registered, supported (W5-H1, bound +0.08; W6-H2); below fused post hoc; §6.5, §6.6
F5	Training a gentle LoRA against the shipped weights recovers its gain	Qwen3-0.6B, 1 seed, share of the bf16 gain: 96% against 84% fused at MLX 4-bit, 99% against 79% at Q3_K_M; the strong LoRA lost 0.158 and 0.115 of R there	Pre-registered for the gentle LoRA, supported (W5-H2, W6-H1); strong LoRA exploratory; §6.5, §6.6
F6	A weight-only predictor meets its pre-registered bar, but the size of the update predicts as well	Held-out MAE of v2: 0.059 [0.028, 0.090] and 0.084 [0.031, 0.146], bar 0.10; update norm with base damage: 0.049 and 0.049; v2 on Qwen3-4B alone: 0.115	Pre-registered, supported (W3-H2, W3-H3, W4-H3, W4-H4); norm model post hoc; §6.7
F7	At a confidence threshold fixed at bf16, the gentle LoRA loses more coverage	Coverage fell further for the gentle LoRA in 36 of 40 cells; in the pre-registered 4-bit test on a Qwen3-1.7B LoRA, Q4_K_M moved coverage by −1.2 points	Exploratory; pre-registered H4 not supported; §6.8
F8	An importance matrix narrows the gap without closing it	Gap $R_{\text{strong}} - R_{\text{gentle}}$ at Q3_K_M without and with a WikiText-2 matrix: 0.215 and 0.146 on Qwen3-0.6B, 0.059 and 0.034 on OLMo-2 1B	Pre-registered (W6-H4), inconclusive: supported for Qwen3-0.6B only; §6.6
C	Measurement repeatability	MLX and GGUF agree on 97.1% to 99.4% of the bf16 predictions of each fine-tune that learned its task; one and four llama-server slots differ on 2 of 3,080 items; decoding restricted to valid labels changes A by at most 0.003	Controls; §3, Appendix B

Proskurina et al. (2024) show that 4-bit quantization lowers a model's confidence in the true label, most for items it was unsure of, and Zhu et al. (2026) show that configuration shifts, including weight quantization, erode the coverage guarantees of conformal prediction. Our operating-point results (Section 6.8) are a single-threshold counterpart.

Pre-registration. Fixing hypotheses, analyses and decision rules before seeing the data separates confirmation from exploration (Nosek et al., 2018), and is still uncommon in machine learning evaluation.

3 Experimental setup

Models and tasks. Table 10 in Appendix B lists the base models, their zero-shot accuracies and the fine-tunes run on each. For banking77 we use PolyAI's original CSV files: the model is fine-tuned on 9,504 training items, 499 are held out for validation (stratified, seed 0), and evaluation uses all 3,080 test items, 40 per intent. The prompt gives an instruction, all 77 intent names and the message, and the Qwen3 chat template is applied with thinking disabled. A prediction is the stripped greedy output and is correct only if it equals the gold intent name exactly. For MASSIVE we use the en-US portion of the original release (11,514 training, 2,033 development and 2,974 test utterances, 60 intents and 55 slot types). The model writes the slot-annotated utterance, which must match the gold annotation after whitespace normalization. Base Qwen3-0.6B scores 0% on this task zero-shot, so on MASSIVE its fine-tunes' whole accuracy is gain. All four models are the released post-trained checkpoints (OLMo-2 1B is OLMo-2-0425-1B-Instruct); "base model" means the model before our fine-tune.

Fine-tuning. Except in one round-6 test (below), all fine-tuning used mlx-lm 0.31.3 for 500 optimizer steps at batch size 4, with the prompt masked from the loss. LoRA used rank 8 and scale 20 on the query, key, value, output, gate, up and down projections of every layer. Gradient checkpointing was on for every full fine-tune and for the 1.7B, OLMo and 4B models. The two Qwen3-4B fine-tunes used micro-batches of 2 with two-step gradient accumulation, after running out of memory at batch size 4; effective batch, steps and examples are unchanged. Figures write learning rates in code notation (1e-5, 1e-4). The gentle setting is mlx-lm's default learning rate at its default LoRA scale of 20; mlx-lm's other defaults differ from our setup (1,000 steps, LoRA on the last 16 layers only, and no prompt masking), and round 6 tests them (Section 6.6). Under Adam a LoRA update grows roughly with the learning rate times the scale, so readers using other toolkits should compare update sizes rather than learning rates: in every pair of rounds 1 to 4, the strong LoRA's update was 8.7 to 12.9 times larger than the gentle one's (Appendix D). Five hundred steps at batch size 4 is 2,000 training examples, about a fifth of one pass over either task (21% of banking77, 17% of MASSIVE). Runs use seed 0 unless marked; second seeds exist for the Qwen3-0.6B banking77 LoRAs at 10^{-5} and 10^{-4} and for the OLMo and MASSIVE LoRAs. One run failed: LoRA at 3×10^{-4} on Qwen3-0.6B never learned the task (final validation loss 1.95, against 0.05 to 0.14 for every other run, and 0% test accuracy), so its gain is undefined and its points are excluded.

Quantization. For MLX, each adapter is fused into the bf16 weights with `mlx_lm fuse` and the fused model is quantized with `mlx_lm convert` (affine, group size 64). For GGUF, the fused bf16 model is converted with llama.cpp's converter and quantized with `llama-quantize`, without an importance matrix except in round 6. Round 1 also evaluated unfused adapters on top of an MLX-quantized base, round 5 trained adapters directly on a quantized base (Section 6.5), and round 6 served LoRA adapters at load time in llama.cpp (Section 6.6). Table 12 in Appendix B gives bits per weight measured from the files. The k-quants spend more bits than their names suggest: Q3_K_M, at 4.2 to 4.4 bits per weight, is closer in size to MLX 4-bit (4.5) than to MLX 3-bit (3.5). Comparisons across engines should therefore use measured size, not the nominal bit count.

Evaluation. Decoding is greedy. For MLX we use our own greedy loop, which casts the final logits to float32 before the log-softmax, because mlx-lm returns bf16 log-probabilities, which are too coarse for confidence scores. GGUF models run in llama-server with four parallel slots. Both engines receive identical token IDs. A prediction's confidence is the exponential of the summed log-probabilities of its generated tokens. As a noise floor, the two engines agreed on 97.1% to 99.4% of the bf16 test predictions of every fine-tune that learned its task, and their accuracies differed by at most 0.61 points. For one fine-tune at Q3_K_M, llama-server with one slot and with four gave different predictions on 2 of 3,080 items (Appendix B). Base models with nonzero accuracy agreed on 95.9% to 98.5%.

Gain retention. For a fine-tune at format v,

$$R(v) = \frac{\mathrm{acc}_{\mathrm{ft}}(v) - \mathrm{acc}_{\mathrm{base}}(v)}{\mathrm{acc}_{\mathrm{ft}}(\mathrm{bf16}) - \mathrm{acc}_{\mathrm{base}}(\mathrm{bf16})}, \tag{1}$$

where the bf16 reference comes from the same engine as v and the base model is quantized to the same format. $R = 1$ means the fine-tune is as far ahead of an equally quantized base as it was at full precision. R exceeds 1 when quantization costs the base more accuracy than the fine-tune, which in our data happened at low bit widths for the bases with substantial zero-shot accuracy (Qwen3-1.7B, OLMo-2 1B and Qwen3-4B; Section 6.1). We therefore also report accuracy retention, $A(v) = \mathrm{acc}_{\mathrm{ft}}(v)/\mathrm{acc}_{\mathrm{ft}}(\mathrm{bf16})$, which does not depend on the base. R is the pre-registered quantity; A was added as an exploratory measure after round 1.

Uncertainty. Intervals for R and for differences in R come from a paired bootstrap over test items (2,000 resamples, seed 0, 95% percentile intervals), with the same resampled items for every model in a comparison. Intervals for a predictor's mean absolute error resample whole fine-tunes, because the points from one fine-tune are not independent. These intervals reflect test-set sampling only, not variation between training runs: on MASSIVE at MLX 3-bit, the difference tested in Section 6.4 has non-overlapping intervals for its two seeds. Exploratory comparisons of accuracy retention use the same paired bootstrap.

Operating point. A confidence gate accepts a prediction when its confidence is at least τ. We fit τ as the lowest threshold at which the full-precision fine-tune's error among accepted items is at most 5%, on the even-indexed test items. Coverage and error among accepted items are then measured on the odd-indexed items, at full precision and at each format, with the same τ. The round-1 plan said later rounds would fit τ on the validation split; they did not, a deviation listed in Appendix A.

4 Measuring what quantization does to an update

Let W be a base weight matrix and W' the corresponding fine-tuned matrix, rounded to the checkpoint's own precision. The update is $\Delta = W' - W$. With Q the exact quantizer of a format (MLX affine with its group size, or llama.cpp's) and D the matching dequantizer, the update that survives quantization is

$$\Delta_q = D(Q(W')) - D(Q(W)). \tag{2}$$

Concatenating Δ and Δ_q over every fine-tuned linear layer, we define

$$\mathrm{kept} = \frac{\langle \Delta_q, \Delta \rangle}{\|\Delta\|^2}, \qquad \mathrm{NSR} = \frac{\|\Delta_q - \mathrm{kept} \cdot \Delta\|}{\|\Delta\|}. \tag{3}$$

Update kept is the share of the update that survives along its own direction. The noise-to-update ratio, NSR, is the size of everything else quantization adds to the difference between the two models, relative to the update itself. That noise comes from rounding at the quantization step, so NSR is in effect the size of the step relative to the update: a high NSR means that quantization moves these weights further than fine-tuning did. Both need only the two sets of weights and the quantizer: no data and no forward pass. Our numpy implementation of MLX's affine quantizer is bit-exact against MLX on the CPU, and GGUF k-quants are computed with llama.cpp's own quantizer.

A third quantity describes the base model rather than the update. Base damage is the mean per-token KL divergence, in nats, of the quantized base model's next-token distribution from its bf16 self, on 8 chunks of 512 tokens of WikiText-2 test text (Merity et al., 2017). It needs one measurement per base model and format, which takes a few minutes for a 1B model. In the supplementary files the three quantities are `signal_retained`, `noise_to_signal` and `KLD_base`.

We use two descriptive labels for a fine-tune and format pair, with thresholds chosen after seeing the round-1 and round-2 data (Table 9). The pair is rounded away when less than 60% of the update survives, and noise-dominated when most of it survives but NSR is 3 or more. These quantities describe the weights. They do not show by themselves that rounding of the update is what costs accuracy; rounds 5 and 6 test that directly (Sections 6.5 and 6.6).

5 Protocol

The study ran in six rounds (Table 5 in Appendix A): four between 27 and 29 September 2026, a fifth on 3 and 4 October, added after a review of the draft pointed to evidence against our first account of the

mechanism, and a sixth from 8 to 10 October, which tested the fifth's remedy in GGUF together with mlx-lm's defaults and an importance matrix. Round n corresponds to the files marked week n in the project repository. Before each round, a pre-registration fixed the configurations, hypotheses, statistical tests and verdict rules, and from round 2 on the consequences of key outcomes. Its SHA-256 hash and the time were recorded in a local log before the confirmatory runs. These records are self-kept. Rounds 3 to 6 also have an outside record: each plan was pushed to a hosted repository before any fine-tune in that round had been evaluated (round 3's after six evaluations of its base model, an hour before its first fine-tune result; round 4's 17 seconds after its evaluation sweep started, which a base-damage measurement had preceded by 7 minutes, and 49 minutes before its first result; round 5's before its run started; round 6's before any of its models was trained). Rounds 1 and 2 have no outside timestamp. The push records can be shared with the action editor. Results that already existed when a plan was written were labeled exploratory: in round 1 the Qwen3-0.6B results were on disk, so only the Qwen3-1.7B runs were confirmatory. Each hypothesis is reported as supported, not supported or inconclusive. Each predictor was fitted by a procedure fixed in advance and frozen, with its hash recorded, before it was scored. Predictor v1 was fitted on round-1 Qwen3-0.6B points only; its Qwen3-1.7B test fine-tune from round 1 had already been evaluated when it was frozen. Deviations were appended to the plan with a timestamp and the file was hashed again. Hypothesis labels follow the pre-registrations: H1 to H4 in round 1, P1 to P3 in round 2, W3-H1 to W3-H3 and W4-H1 to W4-H4 in rounds 3 and 4, W5-H1 and W5-H2 in round 5, and W6-H1 to W6-H4 in round 6.

6 Results

6.1 Round 1: a fine-tune can outlast its base

Table 11 in Appendix B gives the round-1 verdicts for the Qwen3-1.7B LoRA at learning rate 10^{-4}. At 8 and 6 bits, R was 1.01 to 1.07 in both engines (H1). H2 failed in an instructive way: R was higher at 3 bits than at 4, because the 1.7B base collapses under quantization faster than the fine-tune does. At MLX 3-bit the base's accuracy fell from 37.9% to 10.4% and its rate of valid labels from 83.9% to 36.3%, while the fine-tune kept 93% of its own accuracy (98% at Q3_K_M and 85% at Q2_K). $R > 1$ therefore means the fine-tune is further ahead of an equally damaged base, not that it improved, which is why accuracy retention is reported alongside R from here on. At MLX 2-bit both models score 0%. Unfused adapters did not beat fused ones: the interval of the difference included zero at MLX 4-bit and lay below zero at 3-bit (H3, inconclusive). The 4-bit operating point moved on MLX (coverage -4.2 points) but not on GGUF (-1.2), so H4, which required both, was not supported.

The exploratory Qwen3-0.6B runs (Figure 1) showed the pattern the rest of the study tests. The LoRA at 10^{-4} kept 95% of its gain at MLX 3-bit and 101% at Q3_K_M. The LoRA at 10^{-5}, the most accurate of the three at bf16 (78.5%, against 76.6% for full fine-tuning and 73.5% for the LoRA at 10^{-4}), kept 6% at MLX 3-bit, and full fine-tuning at 10^{-5} kept 14%.

6.2 Round 2: within a fine-tuning method, lower learning rates lost more

Round 2 swept the learning rate on Qwen3-0.6B (Table 2; Figure 3 in Appendix B). Learning rate is the variable we manipulated; within each kind of fine-tuning, the update norm $\|\Delta\|$ grows with it. Within LoRA, the learning rate, the size of the update and whatever else a learning rate changes about the solution move together, so this design cannot say which of them matters; the title's "small updates" is shorthand for that bundle. Across kinds of fine-tuning, the norm does not order retention: full fine-tuning at 3×10^{-5} ($\|\Delta\| = 18.6$) kept 0.78 of its gain at MLX 3-bit and 0.89 at Q2_K, while LoRA at 3×10^{-5} (25.2) kept 0.63 and 0.01. Gain retention was positively rank-correlated with learning rate in all eight combinations of fine-tuning kind and format we tracked at 3 and 4 bits: Spearman $\rho = 1.0$ in seven, and 0.8 for LoRA at Q4_K_M, where 3×10^{-5} and 10^{-4} are tied to within 0.001. Each combination has only three or four learning rates, so each is weak evidence alone. The bf16 accuracies did not rank the same way: the most accurate model at full precision, the LoRA at 10^{-5}, kept 6% of its gain at MLX 3-bit. A second seed moved R at MLX 3-bit by at most 0.061 (0.95 and 0.92 for the LoRA at 10^{-4}, 0.06 and 0.00 at 10^{-5}), while bf16 accuracy moved by 8.4 and 3.5 points. On Qwen3-1.7B, the LoRA at 10^{-5} kept 36% of its gain at MLX 3-bit

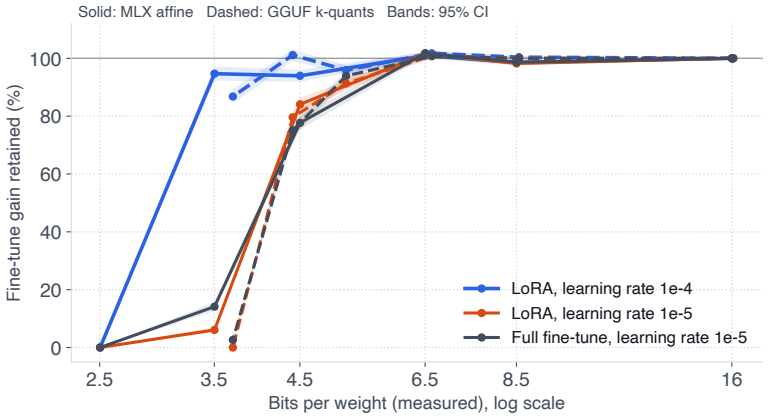


Figure 1: Gain retained by three Qwen3-0.6B fine-tunes on banking77 against measured bits per weight (log scale; 16 is bf16): the LoRA at 10^{-4} (blue), the LoRA at 10^{-5} (orange) and full fine-tuning at 10^{-5} (dark gray). Solid lines: MLX affine; dashed: GGUF k-quants; bands: 95% bootstrap intervals. The strong LoRA holds down to MLX 3-bit; the other two collapse there. Round 1, exploratory.

and the LoRA at 10^{-4} kept 163%. In accuracy retention, which the collapsing base does not inflate, the pair is 0.32 against 0.93.

Table 2: Gain retention R by learning rate, Qwen3-0.6B on banking77, seed 0 (round 2, exploratory). $\|\Delta\|$ is the norm of the update over all fine-tuned linear layers, measured after fusing in bf16. The LoRA at 3×10^{-4} failed to learn and is omitted.

Kind	Learning rate	$\|\Delta\|$	bf16 accuracy	MLX 4-bit	MLX 3-bit	Q4_K_M	Q3_K_M	Q2_K
LoRA	3×10^{-6}	4.0	74.7%	0.75	0.00	0.88	0.45	0.00
LoRA	10^{-5}	9.5	78.5%	0.84	0.06	0.91	0.80	0.00
LoRA	3×10^{-5}	25.2	76.4%	0.91	0.63	0.96	0.91	0.01
LoRA	10^{-4}	87.1	73.5%	0.94	0.95	0.96	1.01	0.87
full	3×10^{-6}	1.0	70.9%	0.27	0.00	0.88	0.19	0.00
full	10^{-5}	4.3	76.6%	0.78	0.14	0.94	0.75	0.03
full	3×10^{-5}	18.6	75.4%	0.93	0.78	0.96	0.98	0.89

6.3 Weight-space correlates of retention

This section is exploratory. Figure 2 in Appendix B plots gain retention against NSR for the round-1 Qwen3-0.6B fine-tunes, and Table 9 counts all 89 fine-tune and format pairs from rounds 1 and 2. When the base is intact and most of the update survives, all 44 pairs with NSR below 3 kept at least 90% of their gain, against 3 of 7 between 3 and 5 and 4 of 10 at 5 or more. The bands were chosen after looking at these data, so the table describes them and tests nothing. Within a fine-tuning method, NSR falls as the learning rate rises, so this association follows from the learning-rate effect itself and is not separate evidence for a mechanism.

The rounded-away mode is rare: five of 192 pairs. Full fine-tuning at 3×10^{-6} produces an update so small, relative to the 3-bit quantization step, that the fine-tuned and base weights round to the same codes: only 5.5% of its update's projection survives MLX 3-bit. Little noise is added because almost nothing is left, so its NSR (2.16) looks benign, and its measured gain retention is 0%, although the base model also collapses there. At MLX 4-bit, where the base is intact, the same fine-tune keeps 50% of its update and 27% of its gain. Across rounds 1 to 4, the five pairs that kept less than 60% of their update were all exploratory

Qwen3-0.6B full fine-tunes, four of them from the run at 3×10^{-6}. No LoRA and no held-out fine-tune came close: the lowest update kept in rounds 3 and 4 was 74%. In 168 of all 192 measured pairs, at least 98% of the projection survived.

Base damage is a second axis. When the base model itself breaks, with base damage of 1 or more (for Qwen3-0.6B and Qwen3-1.7B: MLX 3-bit and 2-bit and GGUF Q2_K), only large updates held on: 4 of 24 such pairs kept 90% of their gain. A large update can carry a broken base: at MLX 3-bit the Qwen3-0.6B base scored 0%, while its LoRA at 10^{-4} kept 95% of its gain. The noise ratio does not separate every case: at Q2_K, the full fine-tune and the LoRA at 3×10^{-5} (Section 6.2) kept 89% and 1% of their gain with almost the same NSR (5.29 and 5.16). One untested explanation is that full fine-tuning also changes the RMSNorm weights, which neither format quantizes.

6.4 Rounds 3 and 4: replication across family, task, seed and size

Rounds 3 and 4 turned the round-2 observation into confirmatory tests. Each test asks whether the LoRA at 10^{-4} keeps more of its gain than the LoRA at 10^{-5}, by a paired bootstrap of $R(10^{-4}) - R(10^{-5})$, at MLX 3-bit and at Q3_K_M. A test is supported only if its 95% interval lies above zero. Round 3 moved to a new model family (OLMo-2 1B on banking77) and a new task (Qwen3-0.6B on MASSIVE); round 4 repeated both with a second seed and added Qwen3-4B. All 10 tests were supported (Table 3), and all ten intervals also exclude zero at the Bonferroni-corrected level of 99.5%. The tests are five fine-tune pairs at two correlated formats, and each interval describes one trained pair. The differences ranged from +0.06 (OLMo at Q3_K_M, with both seeds) to +0.63 (MASSIVE at MLX 3-bit, seed 1). Their sizes are not comparable across settings, because R divides by each fine-tune's own gain, which is smaller for the strong LoRA, and a base that degrades therefore inflates the difference in the strong LoRA's favor. For Qwen3-4B at MLX 3-bit, 0.155 of the 0.539 difference comes from the base's own drop. The last column of Table 3 repeats each test on a quantity that does not involve the base: the gentle LoRA's accuracy drop minus the strong LoRA's (exploratory). It was positive in all ten, by 3.6 to 41.8 points, with every interval above zero.

Table 3: The ten pre-registered tests of whether the LoRA at 10^{-4} keeps more of its gain than the LoRA at 10^{-5}. Difference is $R(10^{-4}) - R(10^{-5})$ with a 95% paired bootstrap interval. Drop is the accuracy lost by the LoRA at 10^{-5} minus that lost by the LoRA at 10^{-4}, in points, with a 95% interval; it does not involve the base and is exploratory. All ten tests were supported.

Test	Setting	Format	R, 10^{-4}	R, 10^{-5}	Difference [95% CI]	Drop [95% CI]
W3-H1	OLMo-2 1B, banking77	MLX 3-bit	1.092	0.826	+0.265 [+0.236, +0.294]	+16.8 [+14.9, +18.7]
W3-H1	OLMo-2 1B, banking77	Q3_K_M	1.076	1.018	+0.058 [+0.037, +0.078]	+3.6 [+2.3, +4.8]
W3-H1	Qwen3-0.6B, MASSIVE	MLX 3-bit	0.621	0.319	+0.301 [+0.271, +0.332]	+20.7 [+18.3, +23.0]
W3-H1	Qwen3-0.6B, MASSIVE	Q3_K_M	0.946	0.843	+0.102 [+0.078, +0.127]	+6.8 [+5.1, +8.4]
W4-H1	OLMo-2 1B, banking77, seed 1	MLX 3-bit	1.105	0.813	+0.292 [+0.263, +0.322]	+18.7 [+16.8, +20.6]
W4-H1	OLMo-2 1B, banking77, seed 1	Q3_K_M	1.064	1.002	+0.061 [+0.042, +0.079]	+3.8 [+2.6, +5.0]
W4-H1	Qwen3-0.6B, MASSIVE, seed 1	MLX 3-bit	0.685	0.052	+0.633 [+0.604, +0.659]	+41.8 [+39.5, +43.8]
W4-H1	Qwen3-0.6B, MASSIVE, seed 1	Q3_K_M	0.916	0.777	+0.138 [+0.111, +0.168]	+9.3 [+7.4, +11.2]
W4-H2	Qwen3-4B, banking77	MLX 3-bit	1.464	0.925	+0.539 [+0.451, +0.642]	+11.8 [+10.0, +13.6]
W4-H2	Qwen3-4B, banking77	Q3_K_M	1.075	0.871	+0.204 [+0.153, +0.256]	+5.1 [+3.9, +6.4]

Table 4 shows accuracy retention for all eight settings of rounds 1 to 4 in which both LoRAs were trained, including the three Qwen3 banking77 settings from rounds 1 and 2; Figure 4 in Appendix B plots its 3-bit columns. In all eight, the gentle LoRA had the higher bf16 accuracy and kept less of its accuracy at each of the five formats compared (MLX 4-bit and 3-bit, GGUF Q4_K_M, Q3_K_M and Q2_K): 40 comparisons in the same direction. MLX 2-bit, evaluated only in round 1, left both Qwen3-0.6B LoRAs at 0%. Two cautions apply. The eight settings are not independent, since three are second seeds of others. And at Q4_K_M the gaps are small: six of the eight are under 2 points, and for two of them, both on MASSIVE, the paired interval includes zero; for a third, OLMo-2 1B with seed 0, it excludes zero by less than 10^{-5}. At MLX 4-bit every gap is at least 3.8 points, and at MLX 3-bit, Q3_K_M and Q2_K at least 4.5.

Ranked by absolute accuracy, which is what a deployment ships, the answer depends on the format (Table 13 in Appendix B). The strong LoRA was the more accurate model at MLX 3-bit and Q2_K in all eight settings and at Q3_K_M in seven, but only in five of eight at MLX 4-bit and in none at Q4_K_M, where the gentle LoRA's higher bf16 accuracy outweighed its larger loss. The strong LoRA's bf16 cost was 1.5 to 9.9 points.

Two checks argue against reading the gentle LoRAs as simply undertrained. Their final validation loss was no higher than the strong LoRAs' in any of the eight pairs. And the gap is not confined to items near a decision boundary: for Qwen3-4B at MLX 3-bit, among the test items each model answered correctly at bf16 with confidence of at least 0.9, the gentle LoRA still answered 89.6% correctly after quantization and the strong LoRA 99.8%. The gentle LoRAs' losses at 4 bits and at Q3_K_M were mostly wrong labels rather than broken outputs: the gentle Qwen3-0.6B LoRA produced a valid intent name for 99.3% of items at MLX 4-bit and 99.5% at Q3_K_M. Broken outputs were 43% of its errors at MLX 3-bit and all of them at Q2_K.

Table 4: Accuracy retention A at the five formats compared. Each cell gives the gentle LoRA (10^{-5}) / the strong LoRA (10^{-4}); the first column gives their bf16 accuracies in percent. Exploratory. †The 95% paired interval of the gap includes zero.

Setting	bf16 accuracy	MLX 4-bit	MLX 3-bit	Q4_K_M	Q3_K_M	Q2_K
Qwen3-0.6B, banking77	78.5 / 73.5	0.88 / 0.98	0.06 / 0.85	0.95 / 1.00	0.79 / 0.98	0.00 / 0.78
Qwen3-0.6B, banking77, seed 1	75.0 / 65.1	0.93 / 0.97	0.00 / 0.82	0.98 / 1.00	0.78 / 0.97	0.00 / 0.77
Qwen3-1.7B, banking77	77.2 / 73.4	0.91 / 0.96	0.32 / 0.93	0.96 / 1.00	0.71 / 0.98	0.00 / 0.85
Qwen3-4B, banking77	84.6 / 78.6	0.94 / 0.99	0.80 / 0.94	0.98 / 1.00	0.93 / 0.99	0.60 / 0.96
OLMo-2 1B, banking77	78.3 / 76.8	0.94 / 0.99	0.76 / 0.97	0.99 / 1.00	0.95 / 1.00	0.55 / 0.95
OLMo-2 1B, banking77, seed 1	79.4 / 77.3	0.93 / 1.00	0.75 / 0.99	0.97 / 0.99	0.94 / 0.99	0.52 / 0.98
Qwen3-0.6B, MASSIVE	65.3 / 62.6	0.93 / 1.01	0.32 / 0.62	0.98 / 0.99†	0.84 / 0.95	0.00 / 0.62
Qwen3-0.6B, MASSIVE, seed 1	62.3 / 54.9	0.96 / 1.04	0.05 / 0.69	0.98 / 0.99†	0.78 / 0.92	0.00 / 0.46

Qwen3-4B differs in degree. Its base does not break at MLX 3-bit (base damage 0.64, against 1.21 for Qwen3-0.6B and 1.08 for 1.7B), and the gap between its two LoRAs is smaller there: accuracy retention 0.80 against 0.94, compared with 0.06 against 0.85 at 0.6B. At Q2_K, where its base does break, the gentle 4B LoRA still loses 40% of its accuracy. With one fine-tune pair per size, these numbers do not establish a trend with size.

Across seeds, OLMo reproduced closely: at MLX 3-bit and Q3_K_M, R moved by at most 0.016 and bf16 accuracy by at most 1.5 points; at other formats R moved by up to 0.042. MASSIVE did not. The direction held, but the gentle LoRA's gain retention at MLX 3-bit went from 0.32 with seed 0 to 0.05 with seed 1, and the strong LoRA's bf16 accuracy fell 7.7 points.

6.5 Round 5: unrounded updates and training on the quantized base

The weight-space account of Section 6.3 has a gap. In round 1 the gentle Qwen3-0.6B LoRA was also evaluated unfused, with its update kept exactly in bf16 on top of the quantized base, and it lost about as much as the fused model: R was 0.83 unfused against 0.84 fused at MLX 4-bit, and 0.02 against 0.06 at 3-bit, where that base collapses. An update that is never quantized cannot be rounded away. These exploratory runs, which our first draft did not report, point to a different account: a small update stops working when quantization moves the weights it was trained against by more than the update itself, whether or not the update is quantized. Round 5 tested that account and the remedy it implies, under a plan pushed to a hosted repository before anything ran (results: Table 7 in Appendix B). A 20-item plumbing test ran after the plan was hashed; its numbers were not used.

W5-H1 asked whether an update that is never rounded rescues a gentle fine-tune. We tested it on OLMo-2 1B at MLX 3-bit, where the base stays usable (14.5% at bf16, 6.8% at 3-bit) while the fused gentle LoRAs kept only 0.83 and 0.81 of their gain. If rounding of the update caused the loss, unfusing should recover most of the missing 0.17 to 0.19; the plan called the hypothesis supported if each seed's upper bound stayed below +0.08. Unfusing changed R by +0.008 [−0.010, +0.027] for seed 0 and +0.023 [+0.002, +0.042] for seed 1,

so W5-H1 was supported: the unrounded update recovered 5% and 12% of the loss by point estimate, and 16% and 22% at the upper ends of the intervals. The strong LoRAs, and both kinds at MLX 4-bit, behaved the same way, with unfused and fused models within 0.015 of each other in R (exploratory).

W5-H2 asked whether training against the shipped base removes the loss. A gentle LoRA trained by mlx-lm directly on the 4-bit Qwen3-0.6B base, the QLoRA setup, and evaluated unfused on that base delivered 96% of the bf16 gain, against 84% for the same LoRA trained in bf16, fused and quantized: a difference of +0.116 [+0.097, +0.135], so W5-H2 was supported. Its accuracy at 4 bits, 77.6%, was within a point of what the bf16-trained LoRA reached at bf16 (78.5%), and its final validation loss was nearly the same (0.057 against 0.056). Exploratory runs at 3 bits and at the strong learning rate qualify the remedy (Appendix B): it helped the gentle LoRA most where the base breaks, cost the strong LoRA at 4 bits, which it already survives, and made no difference to the strong LoRA at 3 bits.

In these MLX tests (OLMo-2 1B at 3 bits, Qwen3-0.6B at 4 bits, banking77), the loss followed the base rather than the update. A gentle update trained against the bf16 weights did not survive their move to the quantized grid, even when the update itself was kept exact, while a gentle LoRA trained against the quantized weights did. The weight measurements of Section 6.3 fit this reading if NSR is taken as the size of the quantization step relative to the update, which is what it tracks; they do not show that the update's own rounding matters. For gentle LoRAs, the remedy is the one QLoRA builds in by training on the quantized base (Dettmers et al., 2023), and that LoftQ prepares for at initialization (Li et al., 2023): train against the weights that will ship.

6.6 Round 6: the GGUF remedy, mlx-lm's defaults and an importance matrix

Round 6 carried round 5's remedy to GGUF and tested two of our setup's choices, under a plan pushed to a hosted repository after plumbing checks and smoke runs on validation items and before any of its models was trained (results: Table 8 in Appendix B). llama.cpp can apply a LoRA at load time on top of a quantized model, so a GGUF adapter can be trained against the weights its file ships: we dequantized every tensor of the Qwen3-0.6B base's Q3_K_M file, trained the gentle LoRA in bf16 on those values with the round-1 settings, converted it to a GGUF adapter and served it on the same file (Appendix B). The converted adapter passed the plan's checks on validation items. All comparisons in the round use round-6 evaluations.

At Q3_K_M the gentle LoRA trained against the shipped weights delivered 0.986 [0.965, 1.009] of the bf16 gain, against $R = 0.795$ [0.771, 0.819] fused and quantized. The bf16-trained adapter applied at load time, its update stored in float32 and never quantized, delivered 0.733 [0.709, 0.757], less than the fused model (a paired difference of −0.062 [−0.082, −0.042], post hoc); training against the shipped weights beat it by +0.253 [+0.229, +0.279], the plan's pre-specified contrast. As the plan predicted, training against the shipped weights recovered more than half of the fused model's loss, exceeding halfway by +0.089 [+0.071, +0.108] (W6-H1 supported), and the load-time adapter recovered less, falling short by 0.164 [0.146, 0.183] (W6-H2 supported). At Q2_K, where the base scores 0%, training against the shipped weights helped both LoRAs; at Q3_K_M it cost the strong LoRA 0.115 of R, as at MLX 4-bit (exploratory; Appendix B).

Trained with mlx-lm's other defaults (1,000 steps, LoRA on the last 16 of 28 layers, no prompt masking), a gentle and a strong Qwen3-0.6B LoRA both learned the task, gaining 66.6 and 73.1 points at MLX bf16, and unlike in our eight settings the LoRA at 10^{-4} was the more accurate one at bf16 (80.7% against 74.2%). It again kept more of its gain, by +0.479 [+0.449, +0.509] in R at Q3_K_M and +0.259 [+0.231, +0.286] at MLX 4-bit (W6-H3 supported); at Q3_K_M the default gentle LoRA kept 0.38 of its gain, against 0.795 for ours. Where the base breaks, neither default LoRA held on: both kept at most 0.4% of their gain at MLX 3-bit and none at Q2_K, where our strong LoRA kept 0.868 (exploratory).

An importance matrix computed on WikiText-2 narrowed the gap on both models without closing it. At Q3_K_M it raised the gentle Qwen3-0.6B LoRA's R from 0.795 to 0.889 and narrowed its gap to the strong LoRA from 0.215 to 0.146, by less than half. On OLMo-2 1B, whose LoRAs both kept their full gain at Q3_K_M (R of 1.02 and 1.08), the gap fell from 0.059 to 0.034 because the strong LoRA's R fell; whether by less than half was inconclusive there, and so was W6-H4, which needed both models.

6.7 Predicting retention before quantizing

Round 2 tested whether the weight measurements predict gain retention without evaluating the quantized fine-tune. Predictor v1, a logistic model of NSR and base damage fitted on the round-1 Qwen3-0.6B points and frozen, met two of its three pre-registered criteria and missed the third, a mean absolute error (MAE) of at most 0.10 on new learning rates and seeds, by 0.007 (Appendix C); as pre-registered, the tool was then limited to descriptive output. Its largest miss was the full fine-tune at 3×10^{-6} at MLX 3-bit, whose update is rounded away: NSR measures only the noise, so it read a vanished update as safe. Predictor v2 adds update kept,

$$\hat{R} = \sigma\left(5.612 + 3.296 \ln \frac{\text{kept}}{\text{NSR}} - 1.555 \ln \text{KLD}_{\text{base}}\right), \tag{4}$$

with its form chosen among four candidates by leave-one-fine-tune-out cross-validation on all 89 round-1 and round-2 points (error 0.071, against 0.095 for the v1 form), and was frozen before round 3.

Predictor v2 passed both of its pre-registered tests in rounds 3 and 4 (Table 15 and Figure 5 in Appendix C): an accuracy bar, MAE at most 0.10 with agreement on the call "keeps at least 90%" of at least 0.85 (W3-H2, W4-H3), and a lower MAE than two baselines, the mean retention of each format and a one-feature model of base damage (W3-H3, W4-H4). Its MAE was 0.059 on round 3 (OLMo-2 1B and MASSIVE, 35 points) and 0.084 on round 4 (second seeds and Qwen3-4B, 42 points), where the interval reaches 0.146, above the bar, and overlaps the base-damage baseline's. Predictor v1 would also have met the accuracy bar in both rounds (0.071 and 0.090). With five or six held-out fine-tunes per round the bars cannot separate the two versions, and no held-out fine-tune entered the rounded-away regime that update kept was added for: the lowest update kept was 74% in round 3 and 98.5% in round 4.

Two baselines that were not pre-registered put these errors in context (Table 15). Predicting full retention everywhere, $\hat{R} = 1$, scores 0.153 in round 3, where v2 beats it by 0.094 [0.003, 0.188] (paired, resampling fine-tunes), but 0.102 in round 4, where the difference, 0.019, has an interval from -0.076 to 0.127. A model of the update's norm and base damage, $\sigma(a + b \ln\|\Delta\| + c \ln \text{KLD}_{\text{base}})$, fitted by the same procedure on the same 89 points, does better than v2: its leave-one-fine-tune-out error is 0.061 against 0.071, its held-out error is 0.049 in both rounds, and in round 4 it beats v2 by 0.035 [0.006, 0.070]. Over the 89 fitting points, $\ln \text{NSR}$ and $\ln\|\Delta\|$ correlate at -0.83 to -0.97 within nine of the ten formats (-0.55 at MLX 3-bit; three formats have only four points). So the weight-space measurements have not been shown to add information beyond the size of the update.

Appendix C describes where v2 misses: on MASSIVE when the base breaks, on banking77 for OLMo-2 and Qwen3-4B, and between seeds that the weights cannot tell apart. On the Qwen3-4B points alone it fell short of both bars, a breakdown that was not pre-registered; after seeing it, we decided that `ftquant check` would not predict for Qwen3-4B.

6.8 Operating-point drift (exploratory)

The one pre-registered test of operating-point drift, round-1 H4 at 4 bits, was not supported, because GGUF Q4_K_M moved coverage by only 1.2 points. At lower bit widths, coverage fell much more than accuracy. For the round-1 Qwen3-1.7B LoRA, MLX 3-bit kept 93% of the accuracy but cut the share of auto-accepted items from 38.1% to 27.8%, and Q2_K kept 85% of the accuracy but cut coverage from 36.4% to 10.1%. Across the eight settings and five formats of Table 4, the gentle LoRA's coverage fell further than the strong LoRA's in 36 of 40 cases; the exceptions are Qwen3-1.7B at Q4_K_M and the second MASSIVE seed at MLX 3-bit, Q4_K_M and Q2_K. At MLX 4-bit, the Qwen3-0.6B LoRA at 10^{-5} kept 88.5% of its accuracy but accepted 19.4 points fewer items. The items a gentle LoRA still accepted were also less reliable: at MLX 3-bit its error among accepted items rose from 5.1% to 12.8% on Qwen3-4B, from 3.8% to 11.4% on OLMo-2 1B (seed 0) and from 5.4% to 23.2% on Qwen3-1.7B, while the strong LoRAs ended at 4.9% to 5.5%. These are point estimates on the half of the test set not used to fit τ; a system that sends low-confidence outputs to a person would see its workload change at formats where accuracy still looks acceptable.

7 Limitations

- **Scale and tasks.** Four base models, none larger than 4B, and only Qwen3-4B above 1.7B, with one seed. Two tasks, both with short outputs scored by exact match; long-form generation may degrade differently.
- **Fine-tuning.** LoRA rank 8 and scale 20 only, 500 steps (about a fifth of an epoch) outside one round-6 test, and the same nominal learning rates at every size. "Gentle" names a learning rate at one LoRA scale, not an update size. The 4B fine-tunes' gains are small (27 points for the gentle LoRA and 21 for the strong), so their R values are noisier. Full fine-tuning was run only on Qwen3-0.6B, with one seed.
- **Rounds 5 and 6.** banking77 only. The remedy was tested on Qwen3-0.6B with one seed per LoRA, in MLX at 4 bits and in GGUF at Q3_K_M; the exact-update test ran on OLMo-2 1B in MLX and on Qwen3-0.6B in GGUF. The GGUF training model held the dequantized weights in bf16, close to but not exactly the values llama.cpp computes with.
- **Formats.** MLX group size 64 only, and no calibration-based methods such as GPTQ or AWQ. GGUF files used an importance matrix only in round 6, computed from general text; one computed from task text could help a gentle LoRA more.
- **Metrics and uncertainty.** R exceeds 1 whenever the base loses more than the fine-tune, which makes it hard to read at 3 bits and below, so we report accuracy retention next to it. Bootstrap intervals cover test-set sampling only, and seed-to-seed variation can be larger (Section 6.4). τ was fitted on half of the test set rather than the planned validation split, and the drift numbers are point estimates.
- **Predictor.** Its round-4 interval crosses the 0.10 bar, it fails on the 4B subgroup, and a post hoc model of update size and base damage did as well or better. Its added input, update kept, targets a failure mode that no held-out fine-tune reached, and its reading bands were chosen post hoc.
- **Process.** The pre-registrations were not reviewed externally, their timestamps are self-recorded except for the pushes of rounds 3 to 6, and all runs used one machine. Round 5 was designed after we had seen the round-1 unfused results that motivated it, and round 6 after round 5's results.

8 Conclusion

At MLX 3-bit and GGUF Q3_K_M, the gentle LoRA kept less of its gain than the strong one in 10 of 10 pre-registered tests across three base models and two tasks, and also at Q3_K_M and MLX 4-bit with mlx-lm's other defaults (F1); it kept a smaller share of its accuracy at all five formats compared, 40 of 40 (F2). The loss did not come from rounding the update: keeping it exact recovered 5% and 12% of the loss in MLX and, post hoc, did worse than fusing in GGUF (F4), while training the gentle LoRA against the weights that ship delivered 96% of the bf16 gain at MLX 4-bit and 99% at Q3_K_M, against 84% and 79% (F5). The weight measurements describe these losses but have not been shown to predict them better than the size of the update (F6). In practice, select a fine-tune on the quantized artifact it will ship as (F3), and if a gentle LoRA must ship at a format where it loses its gain, train it against those weights: on the quantized base in MLX, or on the dequantized file, served as an adapter, in llama.cpp. We tested this on Qwen3-0.6B with one task and one seed per LoRA, and the same step cost the strong LoRA 0.158 of R at MLX 4-bit and 0.115 at Q3_K_M. An importance matrix narrowed the gap without closing it (F8).

Broader Impact Statement

This work measures how quantization changes fine-tuned models that people already ship, and describes a check that flags risky formats before deployment. The main risk is overtrust: the predictor was tested only on the models, tasks and formats listed here, so the tool prints no prediction outside that scope. All data sets and models used are publicly available.

References

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, 2020. arXiv:2003.04807.

Albert Catalan-Tatjer, Niccolò Ajroldi, and Jonas Geiping. Training dynamics impact post-training quantization robustness. *arXiv preprint arXiv:2510.06213*, 2025.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2208.07339.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.14314.

Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2407.09141.

Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. Exploiting LLM quantization. *arXiv preprint arXiv:2405.18137*, 2024.

Kazuki Egashira, Robin Staab, Mark Vero, Jingxuan He, and Martin Vechev. Mind the gap: A practical attack on GGUF quantization. In *International Conference on Machine Learning*, 2025. arXiv:2505.23786.

Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, Swetha Ranganath, Laurie Crist, Misha Britan, Wouter Leeuwis, Gokhan Tur, and Prem Natarajan. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. arXiv:2204.08582.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023. arXiv:2210.17323.

Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. Software, commit 7fe450e19 used in this study, 2023. URL https://github.com/ggml-org/llama.cpp.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. Software, 2023. URL https://github.com/ml-explore/mlx.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

Uygar Kurt. Which quantization should I use? A unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct. *arXiv preprint arXiv:2601.14277*, 2026.

Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. "Give me BF16 or give me death"? Accuracy-performance trade-offs in LLM quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. arXiv:2411.02355.

Tung-Ling Li, Jiale Huang, Lee-Chi Wang, and Janaki Ram Gotei. Scale-QLoRA: Code-invariant adapter merging for native 4-bit microscaling LLMs. *arXiv preprint arXiv:2609.04526*, 2026.

Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, et al. LoftQ: LoRA-fine-tuning-aware quantization for large language models. *arXiv preprint arXiv:2310.08659*, 2023.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024. arXiv:2306.00978.

James Liu, Guangxuan Xiao, Kai Li, Jason D. Lee, Song Han, Tri Dao, et al. BitDelta: Your fine-tune may only be worth one bit. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2402.10193.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. arXiv:1609.07843.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

proflouishyman. mlx_lm fuse produces non-functional model with gpt-oss-20b MXFP4-Q8 (adapter-loaded serving works). GitHub issue ml-explore/mlx-lm #1172, 2026. URL https://github.com/ml-explore/mlx-lm/issues/1172.

Irina Proskurina, Luc Brun, Guillaume Metzler, and Julien Velcin. When quantization affects confidence of large language models? In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. arXiv:2405.00632.

Team OLMo. 2 OLMo 2 Furious. In *Conference on Language Modeling*, 2025. arXiv:2501.00656.

vaibzvb. Fix LoRA/DoRA fuse silently dropping adapter on quantized base (#1172). GitHub pull request ml-explore/mlx-lm #1564, closed without merging, 2026. URL https://github.com/ml-explore/mlx-lm/pull/1564.

Zekun Wu, Swati Dhiman, and Adriano Koshiyama. Which decisions low-bit quantization breaks, and how to predict them. *arXiv preprint arXiv:2608.06564*, 2026.

Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, et al. QA-LoRA: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Zhiwei Zhang, Fali Wang, Xiaomin Li, Zongyu Wu, Xianfeng Tang, Hui Liu, et al. Catastrophic failure of LLM unlearning via quantization. In *International Conference on Learning Representations*, 2025. arXiv:2410.16454.

Yuqicheng Zhu, Jialin Yu, Lin Li, Gengyuan Zhang, Zhen Yang, Steffen Staab, Puneet Dokania, Philip Torr, Jie Tang, and Evgeny Kharlamov. Conformalized large language models under configuration shift. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, 2026. arXiv:2608.01460.

A Pre-registration record

Table 5 lists the six rounds, when each plan was hashed and what it found; Table 6 lists every deviation from the plans. Every amendment changed only a plan's deviation log: in each plan, the text before the deviation section is byte-identical to the version first hashed, and the supplementary material includes those first versions.

Table 5: The six pre-registered rounds. Plan hashed: the time (PDT) at which each plan's hash was recorded.

Round	Plan hashed	Confirmatory question	Outcome
1	27 Sep 17:39	How much of a LoRA's gain survives each format on Qwen3-1.7B?	H1 supported; H2, H4 not; H3 inconclusive
2	27 Sep 18:00	Does predictor v1 predict retention on new fine-tunes?	P2, P3 met; P1 not met
3	28 Sep 08:40	Do small updates lose more on a new family and task? Does predictor v2 pass?	W3-H1 to W3-H3 supported
4	28 Sep 18:11	Do the results hold for second seeds and Qwen3-4B, with v2 unchanged?	W4-H1 to W4-H4 supported
5	3 Oct 19:49	Does an unrounded update rescue a gentle LoRA? Does training on the quantized base?	W5-H1, W5-H2 supported
6	8 Oct 19:43	Does training against the shipped GGUF weights rescue a gentle LoRA where a load-time adapter does not? Does the pattern hold with mlx-lm's defaults, and does an importance matrix leave more than half of the gap?	W6-H1 to W6-H3 supported; W6-H4 inconclusive

Table 6: Deviations from the pre-registered plans, in time order.

When (PDT)	Round	Deviation	Effect
27 Sep, 18:00	1	The analysis code for τ was changed to match the written definition, before any 1.7B result existed.	None on confirmatory results.
27 Sep, 23:48	2	The predictor freeze step crashed on a missing folder before writing anything, and was re-run.	None.
28 Sep, 08:29	2	LoRA at 3×10^{-4} failed to learn, so its R is undefined; its 7 points were excluded from the round-2 test on new learning rates and seeds (P1, P2).	That test was scored on 42 points instead of 49 (Table 14).
29 Sep, 05:16	4	The 4B fine-tunes ran out of memory at batch size 4 and were trained with micro-batches of 2 and two-step accumulation.	Same effective batch, steps and examples.
29 Sep, 16:29	2 to 4	τ was fitted on half of the test set, not the validation split the round-1 plan named for later rounds. Found in an audit.	Exploratory drift numbers only.
4 Oct, 00:55	5	The exploratory run training the LoRA at 10^{-4} on the 3-bit base slowed sharply under memory pressure after its step-100 checkpoint; it was stopped after step 200 and retrained from scratch with the same settings.	None on the confirmatory results.
8 Oct, 23:10	6	The run stopped at the plan's free-disk rule, and again on 9 October at 19:39, both times after its confirmatory phases; before the first stop, free disk fell faster than the run's own files account for. It resumed on 9 and 10 October without repeating any step.	None.

B Additional results

Table 7: Round 5. Top: R of OLMo-2 1B LoRAs at MLX 3-bit, fused and quantized against unfused on the quantized base. Bottom: share of the bf16 gain delivered at the training format by a Qwen3-0.6B LoRA trained on the quantized base (S), against R of the same LoRA trained in bf16, fused and quantized. Differences are paired bootstrap 95% intervals. W5-H1 needed each upper bound below +0.08 and W5-H2 an interval above zero; both were supported.

Test	LoRA	Format	Two pipelines	Difference [95% CI]
Adapter loaded unfused on the quantized OLMo-2 1B base: R fused / R unfused				
W5-H1	10^{-5}, seed 0	MLX 3-bit	0.826 / 0.835	+0.008 [−0.010, +0.027]
W5-H1	10^{-5}, seed 1	MLX 3-bit	0.813 / 0.836	+0.023 [+0.002, +0.042]
exploratory	10^{-4}, seed 0	MLX 3-bit	1.092 / 1.100	+0.008 [−0.006, +0.021]
exploratory	10^{-4}, seed 1	MLX 3-bit	1.105 / 1.108	+0.003 [−0.010, +0.016]
Qwen3-0.6B LoRA trained on the quantized base: R fused / S trained on the base				
W5-H2	10^{-5}	MLX 4-bit	0.841 / 0.957	+0.116 [+0.097, +0.135]
exploratory	10^{-4}	MLX 4-bit	0.940 / 0.781	−0.158 [−0.181, −0.136]
exploratory	10^{-5}	MLX 3-bit	0.061 / 0.951	+0.890 [+0.865, +0.916]
exploratory	10^{-4}	MLX 3-bit	0.947 / 0.940	−0.007 [−0.032, +0.018]

In round 5's exploratory runs (Table 7), training on the quantized base behaved differently by format and learning rate. On the 3-bit base, which scores 0% on its own, a gentle LoRA trained there reached 67.5% and delivered 95% of the bf16 gain, against 6% for the fuse-then-quantize one (a difference of +0.890 [+0.865, +0.916]), although its final validation loss was higher than in bf16 (0.099 against 0.056). At the strong learning rate, training on the quantized base did not help: the strong LoRA trained less well there (final validation loss 0.110 at 4 bits and 0.121 at 3 bits, against 0.067 in bf16) and delivered 78% of the gain at 4 bits and 94% at 3 bits, against 94% and 95% when trained in bf16, fused and quantized: differences of −0.158 [−0.181, −0.136] and −0.007 [−0.032, +0.018]. The most accurate 4-bit and 3-bit models in this comparison were the gentle LoRAs trained on the quantized base.

Two checks from round 5 bear on the earlier rounds. Rebuilt from the round-1 adapter, the gentle LoRA's Q3_K_M file scored 62.1% with four llama-server slots and with one; the two runs differed on 2 of 3,080 items, and each differed from the round-1 run on 3. And restricting decoding to the 77 intent names changed accuracy retention by at most 0.003 (the gentle LoRA at Q3_K_M kept 0.793 of its accuracy with the restriction and 0.791 without; the strong one 0.987 and 0.985), so the gentle LoRA's losses there are wrong labels, not broken outputs. At Q2_K it kept 2% of its accuracy even when forced to name a valid intent.

Round 6 rebuilt each training model for an adapter trained against the shipped weights from the base GGUF file: gguf-py dequantized every tensor, including the shared embedding, and the values were rounded to bf16; all 310 tensors of the rebuilt model matched that dequantization before training. The LoRA was trained with the round-1 settings, rewritten in PEFT form (lora_alpha 160, so that llama.cpp's α/r equals mlx-lm's scale of 20) and converted with llama.cpp's `convert_lora_to_gguf.py` in float32; for all 196 adapted modules, the update read back from the file matched mlx-lm's to within 10^{-15}, relative. llama-server applied the adapter at scale 1 on the file it was trained against. On the 499 validation items, the load-time adapter gave the same predictions as its fused model at bf16 and 99.0% of them on an 8-bit base, and loaded at scale 0 it agreed with the base model on 99.4%. Round-6 evaluations decoded MLX models in batches of 32 and ran llama-server with four slots and without its host-memory prompt cache, settings that can change predictions near ties, so round-6 numbers are compared only with each other. Every Qwen3-0.6B GGUF file in the round, base included, was built around one shared embedding, as the fused fine-tunes are (Appendix F); OLMo-2 1B does not tie its embeddings.

In round 6's exploratory arms (Table 8), at Q2_K, where the base scores 0%, the gentle LoRA kept none of its gain fused or at load time and 0.947 when trained against the shipped weights. Trained against the shipped weights, the strong LoRA did worse at Q3_K_M, where it already keeps its gain, 0.895 against 1.010 (−0.115 [−0.137, −0.094]), as at MLX 4-bit in round 5, and better at Q2_K, 1.024 against 0.868

Table 8: Round 6. Top: share of the bf16 gain S delivered by a Qwen3-0.6B LoRA trained against the dequantized weights of the shipped GGUF file, or by the bf16-trained adapter, both applied at load time, against R of the bf16-trained LoRA fused and quantized. The statistic is $S-(1+R)/2$ for W6-H1 and W6-H2, positive when a treatment recovers more than half of the fused model's loss (W6-H1 needed an interval above zero, W6-H2 one below); the difference between the two treatments' S for the plan's pre-specified contrast; and $S - R$, computed post hoc, for the exploratory rows. Middle: LoRAs trained with mlx-lm's defaults; W6-H3 needed both intervals above zero. Bottom: the gap between the two learning rates without and with an importance matrix; W6-H4 needed the statistic's interval above zero for both models. Brackets are paired bootstrap 95% intervals.

Test	LoRA	Format	Treatment or model	Values	Statistic [95% CI]
Qwen3-0.6B LoRA shipped as GGUF; values are S / R of the bf16-trained LoRA, fused and quantized					
W6-H1	10^{-5}	Q3_K_M	trained on the shipped weights	0.986 / 0.795	+0.089 [+0.071, +0.108]
W6-H2	10^{-5}	Q3_K_M	adapter at load time	0.733 / 0.795	−0.164 [−0.183, −0.146]
contrast	10^{-5}	Q3_K_M	trained minus load time	0.986 / 0.733	+0.253 [+0.229, +0.279]
exploratory	10^{-4}	Q3_K_M	trained on the shipped weights	0.895 / 1.010	−0.115 [−0.137, −0.094]
exploratory	10^{-4}	Q3_K_M	adapter at load time	1.010 / 1.010	+0.000 [−0.015, +0.014]
exploratory	10^{-5}	Q2_K	trained on the shipped weights	0.947 / 0.000	+0.947 [+0.922, +0.975]
exploratory	10^{-5}	Q2_K	adapter at load time	0.000 / 0.000	+0.000 [+0.000, +0.000]
exploratory	10^{-4}	Q2_K	trained on the shipped weights	1.024 / 0.868	+0.156 [+0.131, +0.182]
exploratory	10^{-4}	Q2_K	adapter at load time	0.842 / 0.868	−0.026 [−0.052, +0.001]
Qwen3-0.6B LoRAs trained with mlx-lm's defaults; values are $R(10^{-4})$ / $R(10^{-5})$, the statistic their difference					
W6-H3	both	Q3_K_M	mlx-lm defaults	0.861 / 0.382	+0.479 [+0.449, +0.509]
W6-H3	both	MLX 4-bit	mlx-lm defaults	0.909 / 0.650	+0.259 [+0.231, +0.286]
exploratory	both	MLX 3-bit	mlx-lm defaults	0.004 / 0.000	+0.004 [+0.002, +0.007]
exploratory	both	Q4_K_M	mlx-lm defaults	0.934 / 0.849	+0.085 [+0.063, +0.107]
exploratory	both	Q2_K	mlx-lm defaults	0.000 / 0.000	+0.000 [+0.000, +0.000]
Gap $R(10^{-4}) - R(10^{-5})$ without / with an importance matrix; the statistic is the second gap minus half the first					
W6-H4	both	Q3_K_M	Qwen3-0.6B	0.215 / 0.146	+0.038 [+0.019, +0.057]
W6-H4	both	Q3_K_M	OLMo-2 1B	0.059 / 0.034	+0.005 [−0.011, +0.021]
exploratory	both	Q2_K	Qwen3-0.6B	0.868 / 0.652	+0.218 [+0.191, +0.245]
exploratory	both	Q2_K	OLMo-2 1B	0.497 / 0.347	+0.099 [+0.071, +0.126]

Table 9: Gain retention by measurement band, over the 89 fine-tune and format pairs of rounds 1 and 2 (Qwen3-0.6B and Qwen3-1.7B on banking77). The bands were chosen after seeing these data.

Measurement	Pairs	Kept $\geq 90\%$ of gain	Lowest
base damage < 1, update kept $\geq 60\%$, NSR < 3	44	44	91%
base damage < 1, update kept $\geq 60\%$, NSR 3 to 5	7	3	78%
base damage < 1, update kept $\geq 60\%$, NSR ≥ 5	10	4	45%
update kept $< 60\%$	4	0	0%
update kept $\geq 60\%$, base damage ≥ 1	24	4	0%

(+0.156 [+0.131, +0.182]), whereas at MLX 3-bit in round 5 it made no difference. With mlx-lm's defaults, both LoRAs kept at most 0.4% of their gain at MLX 3-bit and none at Q2_K; at Q4_K_M they kept 0.85 and 0.93, and the strong LoRA was the more accurate (78.2% against 66.9%), unlike in our eight settings. Their update norms were 14.3 and 66.6, a ratio of 4.7 against 8.7 to 12.9 in rounds 1 to 4; the default gentle update was larger than our gentle LoRA's (9.5, Table 16) yet kept less of its gain at Q3_K_M (0.38 against 0.795). On OLMo-2 1B the importance matrix left the gentle LoRA's R at Q3_K_M almost unchanged (1.019 and 1.020) and lowered the strong one's from 1.078 to 1.054. At Q2_K the matrix lifted the gentle Qwen3-0.6B LoRA from 0 to 0.35 of its gain, against 1.00 for the strong one.

Table 10: Base models and tasks. Base accuracy is zero-shot, at bf16 in MLX, on the full test set. Fine-tunes run 500 optimizer steps at an effective batch size of 4, except round 6's two runs with mlx-lm's defaults (1,000 steps).

Base model	Task	Rounds	Base accuracy	Fine-tunes (learning rates)
Qwen3-0.6B	banking77	1, 2, 5, 6	7.4%	LoRA 3×10^{-6} to 3×10^{-4}, full 3×10^{-6} to 3×10^{-5}; LoRA on a quantized base (round 5); LoRA against dequantized GGUF weights, and with mlx-lm's defaults (round 6)
Qwen3-1.7B	banking77	1, 2	37.9%	LoRA 10^{-5} and 10^{-4}
OLMo-2 1B	banking77	3, 4, 5, 6	14.5%	LoRA 10^{-5} and 10^{-4}
Qwen3-0.6B	MASSIVE	3, 4	0.0%	LoRA 10^{-5} and 10^{-4}, full 10^{-5}
Qwen3-4B	banking77	4	57.5%	LoRA 10^{-5} and 10^{-4}

Table 11: Round-1 confirmatory hypotheses: Qwen3-1.7B LoRA at 10^{-4} on banking77. Values are R unless stated; brackets are 95% paired bootstrap intervals.

	Pre-registered criterion	Result	Verdict
H1	$R \geq 0.95$ at 8 and 6 bits, every lower bound ≥ 0.90	MLX 8-bit 1.01 [0.99, 1.04]; MLX 6-bit 1.05 [1.02, 1.08]; Q8_0 1.01 [0.99, 1.03]; Q6_K 1.07 [1.05, 1.10]	supported
H2	R lower at 3 bits than at 4, and $R < 0.5$ at 2 bits	3 vs 4 bits: MLX 1.63 vs 1.25, GGUF 1.75 vs 1.11; 2 bits: MLX 0.00, Q2_K 1.78	not supported
H3	unfused adapter beats fused at MLX 4 and 3 bits	unfused minus fused: 4-bit +0.022 [−0.002, +0.044]; 3-bit −0.038 [−0.073, −0.004]	inconclusive
H4	confidence gate drifts at both 4-bit formats (error +1 pt or coverage ±3 pts)	MLX 4-bit coverage −4.2 pts; Q4_K_M coverage −1.2 pts	not supported

Table 12: Quantization formats and bits per weight (bpw) measured from the files. MLX files were measured for Qwen3-0.6B only; group-64 affine quantization stores a 16-bit scale and bias for every 64 weights, which adds 0.5 bits per weight at any model size.

Engine and format family	Format	bpw, Qwen3-0.6B	bpw, Qwen3-1.7B	Rounds
MLX affine, group 64	8-bit	8.50		1
MLX affine, group 64	6-bit	6.50		1 to 4
MLX affine, group 64	4-bit	4.50		1 to 5
MLX affine, group 64	3-bit	3.50		1 to 5
MLX affine, group 64	2-bit	2.50		1
GGUF k-quants	Q8_0	8.56	8.53	1
GGUF k-quants	Q6_K	6.63	6.59	1 to 4
GGUF k-quants	Q4_K_M	5.15	5.05	1 to 4
GGUF k-quants	Q3_K_M	4.41	4.23	1 to 5
GGUF k-quants	Q2_K	3.70	3.46	1 to 5

Table 13: Absolute accuracy in percent: the gentle LoRA (10^{-5}) / the strong LoRA (10^{-4}), with the more accurate model in bold at each quantized format. The bf16 column is MLX. Exploratory.

Setting	bf16	MLX 4-bit	MLX 3-bit	Q4_K_M	Q3_K_M	Q2_K
Qwen3-0.6B, banking77	78.5 / 73.5	69.4 / **71.7**	4.3 / **62.5**	**74.6** / 73.0	62.1 / **72.2**	0.0 / **57.2**
Qwen3-0.6B, banking77, seed 1	75.0 / 65.1	**69.6** / 62.9	0.0 / **53.1**	**73.7** / 64.7	58.8 / **63.0**	0.0 / **49.9**
Qwen3-1.7B, banking77	77.2 / 73.4	70.1 / **70.4**	24.4 / **68.2**	**74.0** / 73.4	55.0 / **72.0**	0.0 / **62.6**
Qwen3-4B, banking77	84.6 / 78.6	**79.4** / 77.6	67.9 / **73.7**	**83.2** / 78.6	**79.1** / 78.0	51.0 / **75.6**
OLMo-2 1B, banking77	78.3 / 76.8	73.5 / **76.1**	59.5 / **74.8**	**77.6** / 76.9	74.5 / **76.6**	42.8 / **72.7**
OLMo-2 1B, banking77, seed 1	79.4 / 77.3	74.0 / **77.2**	59.6 / **76.1**	**77.4** / 76.3	75.0 / **76.3**	41.8 / **75.7**
Qwen3-0.6B, MASSIVE	65.3 / 62.6	60.8 / **63.1**	20.8 / **38.9**	**63.9** / 62.5	55.1 / **59.5**	0.0 / **38.8**
Qwen3-0.6B, MASSIVE, seed 1	62.3 / 54.9	**60.0** / 56.9	3.3 / **37.6**	**61.8** / 54.8	48.8 / **50.9**	0.0 / **25.6**

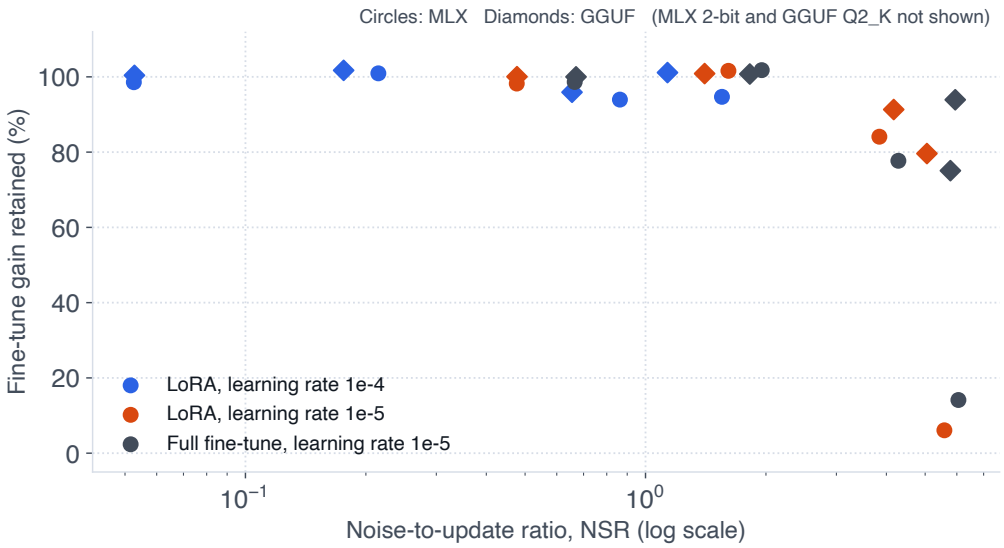


Figure 2: Gain retained against the noise-to-update ratio NSR (log scale) for the round-1 Qwen3-0.6B fine-tunes, at every format except MLX 2-bit and GGUF Q2_K. The base model also breaks at MLX 3-bit; the two points at the lower right are the gentle LoRA and the full fine-tune there. Circles: MLX; diamonds: GGUF.

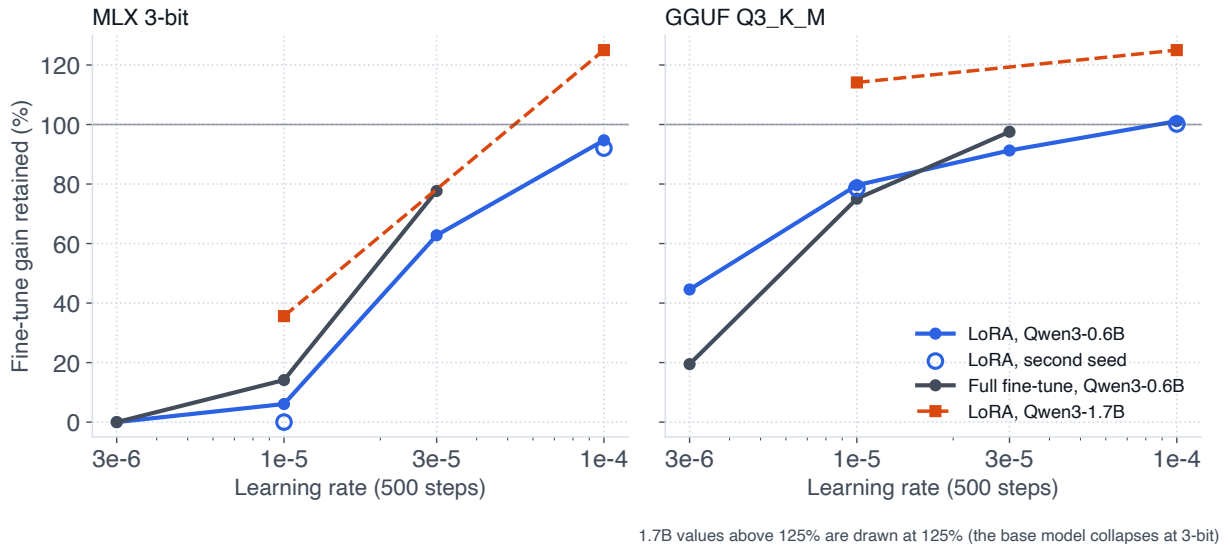


Figure 3: Gain retained at MLX 3-bit and GGUF Q3_K_M against learning rate. Qwen3-1.7B values above 125% are drawn at 125%, because its base collapses at 3 bits. Round 2, exploratory.

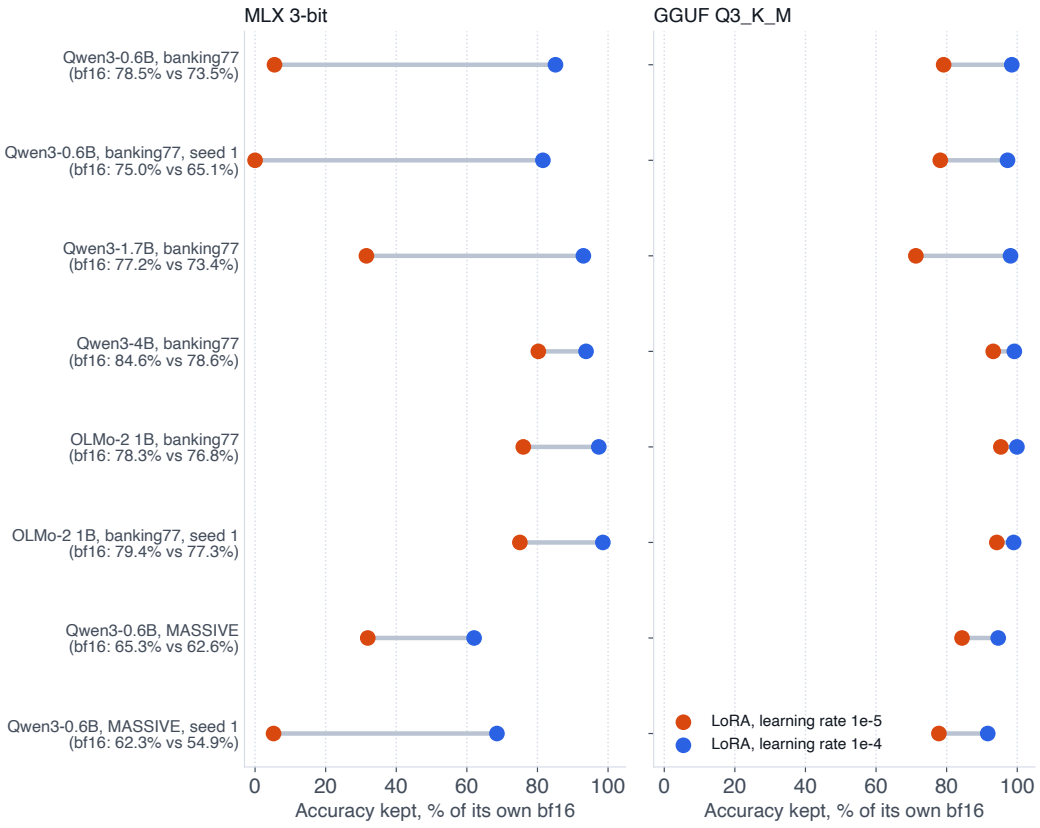


Figure 4: Accuracy kept, as a share of each model's own bf16 accuracy, by the LoRA at 10^{-5} (orange) and the LoRA at 10^{-4} (blue), at MLX 3-bit and GGUF Q3_K_M, in all eight settings. Labels give the two bf16 accuracies. Exploratory.

C Predictor details

Predictor v1 was fitted by least squares against R clipped to $[0, 1]$, on the round-1 Qwen3-0.6B points only, and frozen:

$$\hat{R} = \sigma\left(8.707 - 5.81 \ln \mathrm{NSR} - 2.794 \ln \mathrm{KLD}_{\mathrm{base}}\right). \tag{5}$$

Its test had three pre-registered criteria. On new learning rates and seeds, its MAE had to be at most 0.10, and its agreement with the measurements on the call "keeps at least 90%" at least 0.85 (P1). It had to beat two baselines: the mean retention of each format (bits-only) and a one-feature model of base damage (KLD-only) (P2). On Qwen3-1.7B, its MAE had to be at most 0.15 and its agreement at least 0.80 (P3). P2 and P3 were met; P1 was not, by 0.007, inside an interval from 0.048 to 0.182 (Table 14). In rounds 3 and 4, v1 did almost as well as v2 (0.090 against 0.084 in round 4), and better on round 4's MASSIVE arm (0.072 against 0.088; in round 3, 0.073 against 0.063). Without the easy 6-bit formats, v2's round-4 error was 0.115, against 0.080 in round 3.

Predictor v2's misses in rounds 3 and 4 follow two patterns. On MASSIVE, when the base breaks, the slot-annotation task loses more than a small noise ratio suggests: the LoRA at 10^{-4} at MLX 3-bit was predicted to keep 98% of its gain and kept 62% (seed 0) and 69% (seed 1). On banking77, the OLMo and Qwen3-4B fine-tunes kept more than a large noise ratio suggests: the gentle 4B LoRA kept 92% of its gain at MLX 3-bit with NSR 6.85, where v2 predicted 48%. At Q2_K the same model has $R = 1.28$ against a prediction of 5%, but only because its base collapsed; the fine-tune itself kept 60% of its accuracy. On the 14 Qwen3-4B points alone, v2's MAE was 0.115 and its agreement 0.79, short of both bars. That breakdown was not pre-registered. After seeing it, we decided that `ftquant check` would not predict for Qwen3-4B. Seed-to-seed variation is also invisible to the weights: the two MASSIVE seeds of the gentle LoRA had almost identical measurements and the same 41% prediction at MLX 3-bit, but kept 32% and 5% of their gain.

Table 14: Pre-registered predictor tests. MAE is against R clipped to $[0, 1]$, with 95% intervals from resampling whole fine-tunes. Safe is the share of points where predicted and measured retention fall on the same side of 0.9. The two baseline columns give the MAE of the pre-registered baselines, fitted on the same points as the predictor; their intervals are in Table 15. W3-H2 and W4-H3 are the accuracy bars; W3-H3 and W4-H4 require beating both baselines. *Exploratory breakdown of 2 fine-tunes, without an interval.

Test	n	MAE [95% CI]	Safe	MAE, KLD-only	MAE, bits-only	Result
Round 2, v1: new learning rates and seeds, 0.6B	42	0.107 [0.048, 0.182]	0.88	0.170	0.173	P1 not met; P2 met
Round 2, v1: Qwen3-1.7B	17	0.103 [0.067, 0.154]	0.88	0.147	0.140	P3 met
Round 3, v2: OLMo-2 1B and MASSIVE	35	0.059 [0.028, 0.090]	0.94	0.094	0.128	W3-H2, W3-H3 supported
Round 4, v2: second seeds and Qwen3-4B	42	0.084 [0.031, 0.146]	0.90	0.113	0.170	W4-H3, W4-H4 supported
Round 4, v2: Qwen3-4B only*	14	0.115	0.79	0.129	0.211	below both bars

Table 15: Held-out error of the predictors and baselines in rounds 3 and 4: MAE against R clipped to $[0, 1]$, with 95% intervals from resampling whole fine-tunes, and safe-call agreement. The last two rows were added after the pre-registered tests and are exploratory; the update-norm model was fitted by the same procedure and on the same 89 points as v2.

	Round 3 (35 points)		Round 4 (42 points)	
Model	MAE [95% CI]	Safe	MAE [95% CI]	Safe
Predictor v2 (pre-registered)	0.059 [0.028, 0.090]	0.94	0.084 [0.031, 0.146]	0.90
Predictor v1	0.071 [0.029, 0.107]	0.94	0.090 [0.026, 0.174]	0.90
Base damage only (pre-registered baseline)	0.094 [0.063, 0.123]	0.80	0.113 [0.083, 0.137]	0.69
Format mean (pre-registered baseline)	0.128 [0.081, 0.182]	0.74	0.170 [0.131, 0.210]	0.64
Constant, $\hat{R} = 1$ (post hoc)	0.153 [0.064, 0.242]	0.69	0.102 [0.027, 0.194]	0.79
Update norm and base damage (post hoc)	0.049 [0.018, 0.080]	0.97	0.049 [0.015, 0.084]	0.88

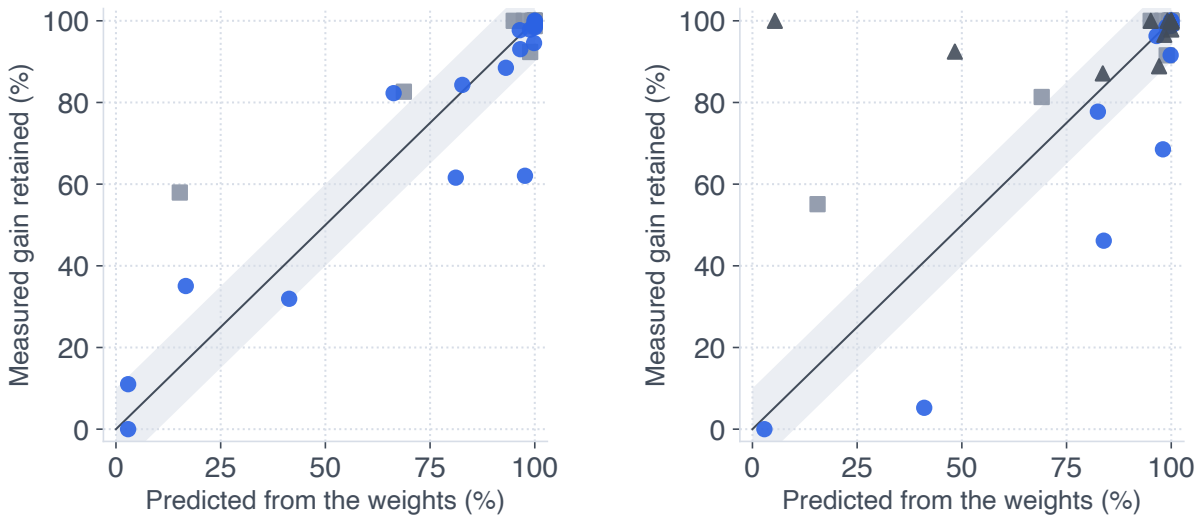


Figure 5: Predictor v2's predictions, made from the weights without evaluating the quantized models, against measured gain retention (capped at 100%). Left: round 3, 35 points (squares: OLMo-2 1B on banking77; circles: Qwen3-0.6B on MASSIVE). Right: round 4, 42 points (squares: OLMo-2 1B, seed 1; circles: Qwen3-0.6B on MASSIVE, seed 1; triangles: Qwen3-4B on banking77). The band marks ±10 points.

D Every fine-tune

Table 16 lists the 23 fine-tunes of rounds 1 to 4, including the run that failed to learn. Together with the base models they account for 279 full-test-set evaluations. Rounds 5 and 6 trained ten more LoRAs, all on Qwen3-0.6B and banking77 (Sections 6.5 and 6.6); their configurations and training logs are in the project repository. $\|\Delta\|$ is the norm of the update over all fine-tuned linear layers, measured on the bf16 fused weights; for round 4 it was computed from the adapters with the same procedure, which reproduces the recorded round-3 values to within 0.2%.

Table 16: The fine-tunes of rounds 1 to 4. Accuracy is at bf16 in MLX. Evaluations counts the full-test-set evaluations, including bf16 in both engines and, in round 1, three unfused-adapter runs.

Round	Base model	Task	Kind	Learning rate	Seed	Accuracy	$\|\Delta\|$	Evaluations	Note
1	Qwen3-0.6B	banking77	full	10^{-5}	0	76.6%	4.3	12	
1	Qwen3-0.6B	banking77	LoRA	10^{-5}	0	78.5%	9.5	15	
1	Qwen3-0.6B	banking77	LoRA	10^{-4}	0	73.5%	87.1	15	
1	Qwen3-1.7B	banking77	LoRA	10^{-4}	0	73.4%	117.0	15	
2	Qwen3-0.6B	banking77	full	3×10^{-6}	0	70.9%	1.0	9	
2	Qwen3-0.6B	banking77	full	3×10^{-5}	0	75.4%	18.6	9	
2	Qwen3-0.6B	banking77	LoRA	3×10^{-6}	0	74.7%	4.0	9	
2	Qwen3-0.6B	banking77	LoRA	10^{-5}	1	75.0%	9.6	9	
2	Qwen3-0.6B	banking77	LoRA	3×10^{-5}	0	76.4%	25.2	9	
2	Qwen3-0.6B	banking77	LoRA	10^{-4}	1	65.1%	88.6	9	
2	Qwen3-0.6B	banking77	LoRA	3×10^{-4}	0	0.0%	335.5	9	failed to learn
2	Qwen3-1.7B	banking77	LoRA	10^{-5}	0	77.2%	11.2	9	
3	Qwen3-0.6B	MASSIVE	full	10^{-5}	0	69.0%	4.4	9	
3	Qwen3-0.6B	MASSIVE	LoRA	10^{-5}	0	65.3%	9.6	9	
3	Qwen3-0.6B	MASSIVE	LoRA	10^{-4}	0	62.6%	83.2	9	
3	OLMo-2 1B	banking77	LoRA	10^{-5}	0	78.3%	11.8	9	
3	OLMo-2 1B	banking77	LoRA	10^{-4}	0	76.8%	113.0	9	
4	Qwen3-0.6B	MASSIVE	LoRA	10^{-5}	1	62.3%	9.5	9	
4	Qwen3-0.6B	MASSIVE	LoRA	10^{-4}	1	54.9%	89.7	9	
4	Qwen3-4B	banking77	LoRA	10^{-5}	0	84.6%	14.9	9	
4	Qwen3-4B	banking77	LoRA	10^{-4}	0	78.6%	192.6	9	
4	OLMo-2 1B	banking77	LoRA	10^{-5}	1	79.4%	12.0	9	
4	OLMo-2 1B	banking77	LoRA	10^{-4}	1	77.3%	122.3	9	

E The `ftquant check` tool

`ftquant check` implements Section 4 for a user's own fine-tune. It reads an MLX or PEFT adapter, or a full checkpoint, streams the safetensors, and runs the exact quantizer on the base and fine-tuned weights. It prints update kept, NSR and base damage. It adds v2's prediction only where v2 was tested: MLX 6, 4 and 3-bit with group size 64 and GGUF Q6_K, Q4_K_M, Q3_K_M and Q2_K, on the three bases whose base damage ships with the tool (Qwen3-0.6B, Qwen3-1.7B and OLMo-2 1B). v2 was fitted on Qwen3-0.6B and Qwen3-1.7B banking77 fine-tunes and tested on held-out OLMo-2 1B, Qwen3-0.6B MASSIVE and Qwen3-4B fine-tunes. The tool needs only numpy. For a Qwen3-0.6B LoRA trained on MASSIVE at 10^{-5}, a fine-tune v2 had never seen, it printed:

```
format                    noise/update  update kept  base damage  predicted gain kept  reading
MLX 8-bit (group 64)              0.47         100%        0.004                  n/a  low noise
MLX 6-bit (group 64)              1.59         100%        0.021                 100%  low noise
MLX 4-bit (group 64)              3.80         100%        0.255                  97%  moderate noise
MLX 3-bit (group 64)              5.52          99%        1.207                  41%  high noise, base model breaks
```

Measured on its 2,974 test items, that fine-tune kept 93% of its gain at MLX 4-bit and 32% at 3-bit. The reading column uses the bands of Table 9. The seed-1 run of the same configuration had the same prediction and kept 5% (Appendix C).

F Reproducibility and compute

Software: MLX 0.32.2 and mlx-lm 0.31.3; llama.cpp build 11146 (commit 7fe450e19), whose `gguf-py` and converter are vendored at the same commit. Data: PolyAI's banking77 CSV files and Amazon's MASSIVE release, both checked against recorded SHA-256 hashes by the fetch script. The project repository holds the six pre-registrations and their hash logs, the frozen predictors (v1 SHA-256 prefix `5df7d620`, v2 `6fd78627`), the per-item predictions of every evaluation, every training configuration and log, and a lab notebook written as the runs happened. Its README maps every table and figure in this paper to the script and the result files that produce it, and `python -m ftquant.findings` recomputes every number quoted in the text, with the source of each, into `runs/findings/findings.json`; repeated runs produce identical files. Re-scoring all results from the per-item predictions needs only Python with numpy and PyYAML.

For Qwen3-0.6B and Qwen3-1.7B, whose checkpoints store a separate copy of their tied output head, the base GGUF files quantized the input embedding at each file's base type, while every fused fine-tune shares one embedding that llama-quantize keeps at Q6_K. Rebuilding the Qwen3-0.6B base with a shared embedding changed its accuracy by at most 0.3 points and lowered its base damage by 2 to 8% at Q4_K_M, Q3_K_M and Q2_K; the Qwen3-1.7B base was not rebuilt. In round 6, every Qwen3-0.6B GGUF file, base included, was built with the shared embedding.

Everything ran on one Apple M1 Pro laptop with 16 GB of memory: rounds 1 to 4 in 51.4 hours of wall-clock time from the first sweep (27 September, 12:45) to the last verdicts (29 September, 16:12), and round 5 in 14.5 hours from 3 October, 20:03, to 4 October, 10:33, several steps of which were slowed by memory pressure on the shared laptop (Appendix A). Round 6 ran for 5.7 hours between 8 October, 19:44, and 10 October, 11:41, in three sessions separated by two stops at the free-disk rule. In rounds 1 to 4, a LoRA run of 500 steps took 10.2 to 19.8 minutes on Qwen3-0.6B, 52.3 to 55.8 on 1.7B, 40.5 to 52.8 on OLMo-2 1B and 127.8 to 143.8 on 4B. One full-test-set evaluation took 2.3 to 22.7 minutes on the smaller models and 12.0 to 25.2 on 4B, and the round-4 mechanism measurements for two fine-tunes took 4.5 to 8.1 minutes on the 0.6B and 1B models and 29.6 on 4B. In round 5, three of the four LoRA runs on a quantized Qwen3-0.6B base took 27.9 to 28.3 minutes; the fourth, retrained after it stalled, took 223.8 minutes of wall-clock time.